
File Handling

Introduction to File Handling

```
def login():
```

```
    password = input()
```

```
    username = input()
```

- Data entered in a program is temporary.
- Once the program execution stops, the data will be lost.
- To store data permanently, we use File Handling.

File Handling allows Python programs to store and retrieve data from external sources.

External Sources

External sources where data can be stored:

- Text File (.txt)
 - CSV File
 - JSON File
 - Excel File
 - Pickle File
 - Database
-

File Handling Operations

File handling is mainly used for:

1. Reading data from a file
2. Writing data into a file
3. Appending data into an existing file

Serialization

Serialization is the process of converting Python objects into a format that can be stored in a file or database.

Deserialization

Deserialization is the process of converting stored data back into Python objects.

Advantage of File Handling

- Data can be stored permanently.
 - Data can be retrieved anytime.
 - Helps in data storage and data processing.
-

Establishing File Connection

Before performing file operations, we must establish a connection between Python program and the file.

Python Module (pl.py) ----- **File Stream/File Object** ----- Text File (file.txt)

open() Function

open() is a built-in Python function used to open a file.

It returns a file object or file connection details.

Syntax: variable = open("file_name", "mode")

1. **File Name:** Name of the file along with extension.data.txt
 2. **Mode:** Specifies the operation to be performed on the file.
-

File Modes**a.Basic file modes are:**

1. Read Mode (r)
2. Write Mode (w)
3. Append Mode (a)
4. Create Mode (x)

b.Special modes are:

1. Read + Write(r+)
2. Write + Read (w+)
3. Append + Read (a+)
4. Create + Write + Read (x+)

c.Binary file modes are:

1. Read binary (rb)
 2. Write binary(wb)
-

Explanation Basic modes:**1.Read Mode (r)**

- Used to read data from a file.
- File must already exist.

Example:

```
f = open("file1.txt","r")  
print(f)
```

Output: <_io.TextIOWrapper name='file1.txt' mode='r' encoding='utf-8'>

Important Points

- Default mode is **read mode**.
- If file does not exist, Python raises: `FileNotFoundError`

Example:

```
f = open("file2.txt")  
print(f)
```

Output: `FileNotFoundError`

Read Mode Operations:

1.read() Method

Used to read data from the file.

Syntax: `file_object.read(size) -> str (string)`

- size → number of characters to read(Optional parameter)

Example:

Assume in Text file(data.txt)

Pyspiders

p1.py

```
f = open("data.txt","r")  
print(f.read())
```

Output: Pyspiders

2.readline() Method

`readline()` is used to read one line at a time from a file.

Syntax: `file_object.readline(size) -> str (string)`

- size → Optional parameter
- Specifies the number of characters to read from the line

- Returns the next line from the file as a string.
- After that it returns an empty string.(because at the end of every line in text file there will be new line character(\n) present.

Example 1 – Basic Example

Text file(data.txt)

Hello Python

Java

C++

p1.py

```
f = open("data.txt", "r")
```

```
print(f.readline())
```

output: Hello Python

Example 2 – Reading Multiple Lines

```
f = open("data.txt", "r")
```

```
print(f.readline())
```

```
print(f.readline())
```

```
print(f.readline())
```

Output:

Hello Python

Java

C++

Each time readline() is called, it reads the next line from the file.

Example 3 – Using Size Parameter

Data.txt: Hello Python Programming Language

```
f = open("data.txt", "r")
```

```
print(f.readline(5))
```

Output: Hello (Only 5 characters are read from the line.)

3. readlines() Method

readlines() is used to **read all lines from a file at once** and return them as a **list of strings**.

Syntax: file_object.readlines(hint) -> list(strs)

- Each element of the list represents **one line from the file**.
- Each line is stored as a **string**.
- The newline character `\n` is included at the end of each line

Example 1 – Basic Example

data.txt

Hello Python

Java

C++

p1.py

```
f = open("data.txt", "r")  
print(f.readlines())
```

Output : ['Hello Python\n', 'Java\n', 'C++']

4. readable() Method

> It is used to check whether a file is readable or not.

> It returns True if the file can be read, otherwise False.

Syntax: file_object.readable() -> bool (Boolean)

- True → File is readable
- False → File is not readable

Example 1 – File in Read Mode

```
f = open("data.txt", "r")  
print(f.readable())
```

Output: True (Because the file is opened in read mode)

Example 2 – File in Write Mode

```
f = open("data.txt", "w")  
print(f.readable())
```

Output: False

Because the file is opened in write mode, it cannot be read.

tell() Method

It is used to return the current stream / file pointer position.

Syntax: file_object.tell() -> int

Example 1 – Initial Pointer Position

data.txt : Pyspiders Institute

```
f = open("data.txt", "r")  
print(f.tell())
```

Output: 0

When a file is opened, the **pointer starts at position 0**.

Example 2 – After Reading Data

```
f = open("data.txt", "r")  
print(f.read(5))  
print(f.tell())
```

Output :

Pyspi

5

After reading **5 characters**, the pointer moves to **position 5**

seek() Method

It is used to move the file pointer or stream position to a specified position in the file.

It allows us to change the current position of the file pointer.

Syntax: file_object.seek(offset) -> int (returns the new file pointer position)

Offset: Number to move the pointer

Example 1

data.txt: Pyspiders Institute

```
f = open("data.txt", "r")
f.seek(5)
print(f.read())
```

Output: ders Institute

The file pointer moves to **position 5** and reading starts from there.

Example 2

```
f = open("data.txt", "r")
print(f.read(5))
print(f.tell())
f.seek(0)
print(f.read(5))
```

Output:

Pyspi

0

Pyspi

seek(0) moves the file pointer back to the **beginning of the file**.

close() Method

It is used to close the file connection after completing file operations.

When a file is closed, the connection between the program and the file is terminated.

Syntax: file_object.close() -> None

Example:

```
f = open("data.txt","r")
print(f.read())
f.close()
```

After calling close(), the file cannot be used for further operations unless it is opened again.

Why Closing File is Important

- Releases system resources
 - Prevents data corruption
 - Improves program efficiency
-

Closed:

>It is a **file object attribute** used to check **whether a file is closed or not**.

>It returns **True if the file is closed** and **False if the file is still open**.

Syntax: file_object.closed -> bool (Boolean)

- True → File is closed
False → File is open

Example

```
f = open("data.txt","r")
print(f.closed)
f.close()
print(f.closed)
```

Output: False

True

Note: closed is an attribute, not a method.

with Statement (With Block)

>The with statement is used to open a file and automatically close it after the block of code is executed.

>It is the recommended way to handle files in Python because it manages the file safely.

Syntax: with open("file_name","mode") as file_object:
#statements

>The open() inside the with statement returns a **file object (TextIOWrapper)**.

Example

```
with open("data.txt","r") as f:  
    print(f.read())
```

2. Write Mode (w)

Write mode is used to **write data into a file**.

Syntax: file_object = open("file_name","w")

>If the file **does not exist**, Python **creates a new file**.

>If the file **already exists**, the **previous content will be deleted** and new data will be written.(data overwritten takes place)

Write Mode Operations:

1.write() Method:

write() is a file method used to write a string to a file.

Syntax: file_object.write(string) -> int(Returns the number of characters written.)

Example:

data.txt: Java

p1.py

```
f = open("data.txt","w")  
f.write("Python")  
f.close()
```

Output: data.txtPython

2.writelines() Method

It is used to write multiple lines into a file at once.

Syntax: file_object.writelines(list_of_strings) -> None

Example:

```
f = open("data.txt","w")
data = ["Python\n","Java\n","C++"]
f.writelines(data)
f.close()
```

Text File Output

Python

Java

C++

3.writable() Method

>It is used to check whether a file can be written to or not.

>If the file is opened in a write mode (w, a, w+, a+), it returns True.

>If the file is opened in read mode (r), it returns False.

Syntax: file_object.writable() -> bool

True → File is writable

False → File is not writable

Example1:

data.txt: Java

Python File

```
f = open("data.txt","w")
print(f.writable())
f.close()
```

Output: True

Since the file is opened in **write mode**, writing operations are allowed, so `writable()` returns **True**.

Example 2:

```
f = open("data.txt","r")
print(f.writable())
f.close()
```

Output: False

Ex 3:Common Example

```
f = open("data.txt","w")
f.write("Python Programming")
print(f.tell())
f.seek(0)
f.write("Java")
print(f.tell())
f.close()
```

Text File Output (data.txt)

Javaon Programming

3.Append Mode (a)

>Append mode is used to add new data at the end of an existing file.

>If the file does not exist, Python creates a new file.

>If the file already exists, the existing content will not be deleted and new data will be added at the end of the file.

Syntax: file_object = open("file_name","a")

Example: write()

Text File Before Execution (data.txt): Python

Python File

```
f = open("data.txt","a")
f.write("Java")
f.close()
```

Text File After Execution: PythonJava

The new data is added at the end of the file without deleting the existing content.

Example:writelines()

```
f = open("data.txt","a")
data = ["HTML\n","CSS\n","JavaScript\n"]
f.writelines(data)
f.close()
```

Example:writable()

```
f = open("data.txt","a")
print(f.writable())
Output: True
```

Example:tell()

```
f = open("data.txt","a")
print(f.tell())
f.close()
```

Example:seek()

```
f = open("data.txt","a")
f.seek(0)
f.write("Python")
f.close()
```

Even if seek(0) is used, data will still be written at the end of the file because append mode always writes at the end.

4.Create Mode (x)

- > Create mode is used to create a new file.
- > If the file does not exist, Python creates the file.
- > If the file already exists, Python raises FileExistsError.

Syntax: file_object = open("file_name","x")

Example 1:

```
f = open("file1.txt","x")
f.write("Python")
f.close()
```

Text File Output (file1.txt)

Python

A **new file is created** and the data is written into the file.

Example 2:

```
f = open("file2.txt","x")
data = ["Python\n","Java\n","C++"]
f.writelines(data)
f.close()
```

Text File Output: file2.txt

Python

Java

C++

Example 3:

```
f = open("file3.txt","x")
print(f.writable())
f.close()
```

Output: True

Example 4:

```
f = open("file4.txt","x")
f.write("Python")
print(f.tell())
f.close()
```

Output: 6

After writing **Python**, the pointer moves to **position 6**.

Example 5:

```
f = open("file5.txt","x")
f.write("Python Programming")
f.seek(0)
f.write("Java")
f.close()
```

Text File Output: Javaon Programming

seek(0) moves the pointer to the **beginning of the file**, and writing starts from that position.

Special modes:**1.r+ Mode:**

r+ mode is used to read and write the data in a file.

1. The file must already exist.
2. It is used for both reading and writing operations.
3. The file pointer starts from the beginning of the file.
4. When we write data, it overwrites the existing content from initial position.
5. If file not exist raises FileNotFoundError.

Syntax: file_object = open("filename","r+")

Example

sample.txt

Python Programming Language

Python Program

```
f = open("sample.txt","r+")
f.write("Java")
f.close()
```

Output in sample.txt

Javaon Programming Language

Example with tell() and seek()

sample.txt: Programming Language

```
f = open("sample.txt","r+")
f.seek(3)
f.write("Java")
f.close()
```

Output sample.txt ProJavaming Language

2.w+ Mode

w+ mode is used for both writing and reading in a file.

1. It is used for write and read operations.
2. If the file already exists, the old content will be deleted.
3. If the file does not exist, a new file will be created.
4. The file pointer starts at the beginning of the file.
5. After writing, we should use seek() to read the content.

Syntax: file_object = open("filename", "w+")

Example

sample.txt contains: Hello

```
f = open("sample.txt","w+")
f.write("Python")
print(f.read())
f.close()
```

Output: empty string

Example with tell() and seek()

```
f = open("sample.txt","w+")
print(f.tell())
f.write("Python")
print(f.tell())
```

```
f.seek(0)
print(f.read())
f.close()
```

Output:

```
0
6
Python
```

3. a+ Mode

a+ mode is used for appending and reading data in a file.

1. It is used for both append and read operations.
2. If the file does not exist, a new file will be created.
3. The file pointer is placed at the end of the file.
4. The new data will be added at the end of the file.
5. To read the content, we should move the pointer using seek().

Syntax: file_object = open("filename", "a+")

Example:

Assume **sample.txt** contains:

Python

p1.py

```
f = open("sample.txt", "a+")
f.write(" Programming")
f.seek(0)
print(f.read())
f.close()
```

Output: Python Programming

Example with tell() and seek()

Assume **sample.txt** contains: Python

```
f = open("sample.txt", "a+")
print(f.tell())
f.write(" Language")
print(f.tell())
f.seek(0)
print(f.read())
f.close()
```

Output

```
6
15
Python Language
```

4. x+ Mode

x+ mode is used to create a new file and perform both reading and writing operations.

1. It is used for creating a new file.
2. If the file already exists, it will raise an `FileExistsError`.
3. It supports both read and write operations.
4. The file pointer starts at the beginning of the file.
5. After writing, we should use `seek()` to read the content.

Syntax: `file_object = open("filename", "x+")`

Example

```
f = open("file2.txt", "x+")
f.write("Python Programming")
f.close()
```

A **new file(file2.txt)** is created and the data is written into the file.

Example with tell() and seek()

```
f = open("sample.txt","x+")  
print(f.tell())  
f.write("Python")  
print(f.tell())  
f.seek(0)  
print(f.read())  
f.close()
```

Output

0

6

Python
